

Energy-Accuracy Tradeoffs in Spiking Neural Networks: Benchmarking Loihi 2 vs. GPU for Sustainability Focused Time-Series Forecasting

Souvik Pramanik and Yang Yi

Abstract— Neuromorphic computing has emerged as a promising paradigm for energy-efficient, parallel, and adaptive information processing. While Spiking Neural Networks (SNNs) offer biologically inspired event-driven computation, their application to regression-style time-series forecasting remains underexplored. In this work, we propose a Ring Buffer-driven SNN (RingSNN) architecture for time-series forecasting. The proposed framework is implemented on Intel’s Loihi 2 neuromorphic hardware and benchmarked against traditional Artificial Neural Networks (ANNs) and Hybrid Deep Neural Network (DNN), which is a hybrid of CNN-LSTM, executed on Nvidia-SMI GPU platforms. To ensure fair comparison, we establish a reproducible energy vs. accuracy benchmarking protocol that accounts for idle, active, and spike-driven power states. Extensive experiments on time-series datasets demonstrate that RingSNN achieves forecasting accuracy comparable to ANN baselines, as measured by Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE), while significantly reducing energy consumption. These results highlight the potential of RingSNN as a viable solution for real-time forecasting in energy-constrained environments such as IoT and edge computing, advancing sustainable neuromorphic applications.

Keywords—Neuromorphic Computing, Spiking Neural Networks (SNN), Ring Buffer-Driven SNN (RingSNN), Artificial Neural Networks (ANN), Intel Loihi 2, Time-Series Forecasting, Energy Efficiency, Benchmarking

I. INTRODUCTION

Recent advances in deep learning architectures have significantly improved time-series forecasting by enabling the modeling of complex temporal dependencies and perfecting prediction accuracy. Among the neural architectures explored in our work, Spiking Neural Networks (SNNs) have emerged as an important upcoming approach. In SNNs, neurons emit spikes only when the membrane potential exceeds a certain threshold. They are designed to process temporal, sequential data through event-driven spiking activity, making them well-suited for capturing dynamic patterns and long-term dependencies in time-series signals [1]. They primarily handle temporal changes, operate efficiently on sparse events, and capture both short-term and long-term dependencies. Their ability to operate on infrequent events combined with energy efficiency makes them particularly attractive for real-world forecasting tasks on resource-constrained devices.

Recent advances in neuromorphic computing have further improved capabilities of SNNs. One such Neuromorphic Computing hardware used is Intel Loihi 2. Loihi 2 is the latest generation of Intel Loihi’s neuromorphic processor family, designed to support event-driven spiking neural networks with customizable on-chip learning mechanisms. This hardware is known for brain-inspired computing principles and achieves high energy efficiency. This platform enables various forms of on-chip learning

such as supervised, unsupervised, and reinforcement through programmable synaptic update mechanisms like spike-timing-dependent plasticity (STDP) [2]. But Davies et. al. in [2] focused on classification and optimization problems and did not explore continuous-valued prediction tasks like forecasting. The above work was architectural and foundational but was neither application-driven nor benchmark oriented. Other research using Loihi processors has demonstrated significant gains in speed and energy efficiency for relevant workloads compared to conventional CPUs and GPUs [3]. However, the authors in [3] did not extend into forecasting with benchmarking.

Recent work has emphasized the lack of comprehensive benchmarking on hardware platforms. Bhattacharjee et. al. in [4] challenges prevailing assumptions about the energy efficiency of SNNs in their work, showing that algorithmic claims overlook the bottlenecks like hardware limitations and memory access. They used two hardware benchmarking platforms: SATA, a sparsity-aware training accelerator, and SpikeSim, an In-Memory Computing (IMC) analog crossbar simulator. The authors show that many algorithmic claims of energy savings in SNN are based on idealized metrics that overlook practical bottlenecks such as memory access and hardware inefficiencies. Their findings emphasize the need for hardware-specific protocols and caution against relying solely on algorithmic sparsity or spike-count metrics. They show that energy claims are often based on Floating-Point Operations per Second (FLOPs), timesteps, or spike sparsity ignoring hardware overheads like memory access and communication or decomposed energy states like idle, active, and spike-driven [4]. In contrast, our work focuses on application-driven benchmarking. We deploy decoder pipelines on Loihi 2 Neuromorphic Hardware, replaying weights across epochs for prediction. This enables meaningful comparisons between Loihi-style SNNs and GPU-based ANNs, especially in time-series forecasting tasks where temporal dynamics and energy vs accuracy trade-offs are critical. The authors in [4] highlights systemic hardware challenges, but did not demonstrate how neuromorphic platforms perform in practice for real-world predictive tasks.

Similarly, the authors in [5] conduct a comprehensive evaluation of energy efficiency across spiking and non-spiking models using real hardware measurements. Their analysis spans multiple platforms including neuromorphic chips and conventional accelerators and reveals that energy advantages of SNN are highly context-dependent, often diminishing when memory access, control overhead, and hardware constraints are considered. That work also demonstrates the need for hardware-aware benchmarking and challenges the notion that spike sparsity alone guarantees energy savings. Their methodology complements our unified benchmarking pipeline by reinforcing the importance of platform-specific energy utilization, per

sample normalization, and task-oriented evaluation. We extend this perspective to time-series forecasting, where temporal dynamics and decoder alignment further influence energy vs accuracy trade-offs across Loihi-style SNNs and GPU-based ANNs.

Recent work in [6] by Nagy provides a compelling demonstration of Loihi 2 energy efficiency in real-time autonomous driving tasks. Their study benchmarks a SNN deployed on Loihi 2 against GPU-based models, showing up to 16 times lower energy consumption while maintaining comparable accuracy. Their methodology reinforces the importance of platform specific energy profiling and validates the feasibility of fair comparisons across neuromorphic and conventional architectures. We build on this foundation by extending the benchmarking scope to time-series forecasting tasks and integrating both Loihi-style SNNs and GPU-based ANNs into a unified architecture. Whereas Nagy [6] validated Loihi 2's efficiency in autonomous driving, our contribution extends neuromorphic benchmarking to the sustainability domain, demonstrating how Loihi 2 can support energy constrained predictive tasks beyond mobility.

Lv et al. in [1] proposes a principled framework that addresses key challenges in deploying SNNs for temporal prediction. Their work handles issues such as temporal alignment, spike encoding complexity, and the lack of standardized model selection criteria [1]. These are barriers that often hinder reproducible benchmarking. By introducing structured design choices and demonstrating competitive performance on time-series datasets, the authors validate the applications of SNNs in forecasting tasks. This complements our goal of building a unified benchmarking pipeline that includes Loihi-style SNNs and GPU-based ANNs. It highlights the importance of encoding strategies, decoder alignment, and task-specific evaluation metrics. Their insights reinforce the need for hardware-aware, reproducible comparisons across neuromorphic and conventional platforms. Lv [1] provides general algorithmic advances and guidelines for SNN forecasting. Our contribution demonstrates practical deployment on neuromorphic hardware for a sustainability focused predictive task, offering per epoch runtime, estimation of spike-based energy consumption, and accuracy metrics.

Prior work on SNNs has highlighted hardware bottlenecks [7], theoretical efficiency bounds ([5], [8]), algorithmic innovations such as pruning and hybrid architectures [34], and application benchmarks in domains like self-driving [6] and time-series forecasting [1]. Davies in [2] introduced the Loihi neuromorphic processor and demonstrated its on-chip learning capabilities, but their focus was primarily on classification and optimization tasks rather than regression-style forecasting. In [8], the authors evaluate SNN energy efficiency but focus on total energy consumption rather than decomposed energy states such as idle, active, and spike-driven.

Overall, prior research has concentrated on classification tasks, optimization problems, continuous-valued predictions, theoretical energy bounds, memory-access, and hardware limitation issues. None have benchmarked regression-style forecasting on Loihi 2 with energy measurement protocols that explicitly account for decomposed energy states. This paper addresses this gap by

presenting hardware-validated, multi-dataset benchmarking of decoder pipelines on Intel Loihi 2 neuromorphic chip, alongside GPU baselines monitored using the Nvidia-SMI utility, for sustainability-focused forecasting tasks such as solar power production and energy consumption.

By replaying weights across epochs and logging runtime, spike counts, and accuracy, we generate reproducible energy vs. accuracy trade-off plots that support practical hardware deployment. The unified benchmarking pipeline spans Loihi 2 neuromorphic SNNs and GPU-based ANNs for time-series forecasting, leveraging Ring Buffer-based streaming of encoded data for continuous prediction. The decoupled decoder design enables analysis of spike-driven representation quality and layer-level energy characteristics, supporting hardware-validated evaluation of neuromorphic efficiency in renewable energy forecasting.

The major contributions of this work are:

1. We design and implement a Ring Buffer-based Spiking Neural Network (SNN) pipeline tailored for regression-style time-series forecasting, enabling efficient temporal encoding and continuous prediction.
2. We propose a reproducible benchmarking methodology for jointly evaluating predictive accuracy and energy consumption in SNNs, enabling comparison between neuromorphic hardware (Loihi 2) and conventional GPU platforms.
3. We introduce an energy measurement protocol that accounts for the addition of idle, active, and spike-driven power states, ensuring fair and reproducible comparisons between Loihi 2 and GPU implementations.
4. We establish that Ring Buffer SNNs are a viable solution for real-time forecasting in energy-constrained environments like IoT and edge devices by combining biologically inspired temporal memory with hardware-specific evaluation.

The remainder of this work is arranged as follows: Section II begins with the motivation for using Ring Buffer in SNNs. This is followed by the description of the datasets used in this work. We later describe our Ring Buffer-driven SNN model including the Dense and LIF layers and their associated equations. The Ring Buffer-driven SNN is compared to the baseline ANN models (ANN and Hybrid CNN-LSTM) and their architectures are described. Section III outlines the different evaluation methods used for assessing performance of the Ring Buffer-Driven SNN and ANN models and their respective hardware implementations using Loihi 2 and Nvidia-SMI GPU. In addition, the concept of Hebbian Weights is discussed for use in the RingSNN Decoder Weights plot. An explanation is provided for the eventual comparison ratios of the ANN architectures to the Ring Buffer-Driven SNN ones. Finally, the evaluation section discusses how different decomposed power states: idle, active, and spiking are calculated. Section IV presents the performance of the models using MAE, RMSE and Efficiency Ratio. In addition, visualization plots like Spike Energy vs. MAE Tradeoff, Spike Raster, Spike Density per

Neuron, Decoder Weights per Neuron, and Ground Truth Plots and Efficiency Ratio Tables are presented. The Ring Buffer-Driven SNN architecture is compared to existing published SNN architectures. Finally, Section V summarizes our findings, draws conclusions, and provides future trajectories of this work.

II. PROPOSED APPROACH

A. Rationale for using Ring Buffers in SNN Model

Ring Buffer is a data structure that works efficiently by continuously overwriting old data by new ones when buffer is full [39]. This means that instead of growing dynamically or requiring shifting of elements, the buffer efficiently reuses its allocated memory. In the context of neural networks, Ring Buffers are often employed in roles for managing and processing data, particularly in scenarios involving streaming data or limited memory resources. Ring Buffers have been used in Neural Networks for the following processes.

- **Data Preprocessing and Augmentation:** When training a neural network with a continuous stream of data like in sensor readings, audio, video frames, a ring buffer can temporarily store a window of recent data for pre-processing before feeding it to the network. This ensures the network always sees the most recent and relevant data without requiring large memory allocations [10].
- **Experience Replay in Reinforcement Learning:** In reinforcement learning, ring buffers are commonly used as experience replay buffers. These buffers store past experiences such as state, action, reward, and next state that an agent has encountered [35]. During training, the agent samples mini-batches of experiences from buffers to learn from a diverse set of past interactions, improving stability and convergence [11].
- **Logging and Monitoring:** Ring buffers can be used to store recent logs or performance metrics related to the neural network's operation, allowing for efficient monitoring and debugging without overwhelming system resources with excessive log storage [12].
- **Real-time Inference with Limited Memory:** In embedded systems or devices with limited memory, a ring buffer can manage the input data for real-time inference, ensuring that only the necessary recent data is held in memory for processing by the trained neural network [13].

The main advantages of using Ring Buffer are its fixed memory footprint and efficient First-In-First-Out (FIFO) behavior with automatic overwriting of old data, which are crucial for performance and resource management [8]. Ring Buffers have been used in classification or event-based tasks [12]. We have adapted this Ring Buffer implementation for regression-style time-series forecasting. The rationale behind this is to have tight control over inputs and spike fluctuations in our SNN implementation on hardware. It maintains a biologically inspired short-term memory of recent spike histories. This rolling buffer enables the network to capture temporal dependencies essential for continuous-valued predictions, while simultaneously

smoothing event-driven spike trains into real-valued outputs suitable for regression tasks. In contrast to conventional deep learning models, which often rely on hidden states with high computational and memory demands, Ring Buffer SNNs achieve efficient temporal representation through sparse, event-driven updates. This design not only aligns with neuromorphic principles of energy-efficient computation but also supports real-time forecasting in streaming environments where low latency and reduced power consumption are critical. Because our work is focused on energy benchmarking and reproducibility, Ring Buffer is a great candidate for energy profiling and benchmarking utility.

B. Datasets

Three datasets are used to examine the performance of the RingSNN model on Loihi 2 and the ANN models on GPU. These datasets are the National Renewable Energy Laboratory (NREL) solar generation profile referred to as the Solar Production [14], PJM Interconnection LLC (PJM) Hourly Energy Consumption [15], and Solar Influences Data Analysis Center (SIDC) Monthly Mean Total Sunspot Number datasets [16].

The Solar Production dataset was taken as part of the ARPA-E PERFORM project. NREL has produced actual and forecasted time-series datasets with load, wind, and solar generation profiles [14]. Both actual and forecasted data have high temporal and spatial fidelity and the forecasted data includes both deterministic and probabilistic forecasts. The datasets are provided for four U.S. Independent System Operator territories: ERCOT, NYISO, MISO, and SPP [14]. We use ERCOT solar production data (5-minute resolution, 2017). The Solar Production levels for the ERCOT dataset are gathered from individual solar facilities at the site level. The input `time_index` represents the time stamp. At each time stamp, the actual solar production on the ERCOT sites is gathered and measured in megawatts (MW) or kilowatt hours (kWh) which is the target or output variable. The dataset has a clean time-series format with no need for complex preprocessing and fits well with ANN and RingSNN models [14]. Furthermore, it has consistent sampling with uniform time intervals making the dataset easier to batch, normalize, and encode. Finally, realistic solar generation patterns are captured including variability and ramp events and are useful for benchmarking energy vs. accuracy tradeoffs in neuromorphic vs. conventional models.

In addition, we also used PJM Hourly Energy Consumption dataset which provides estimated energy consumption for every hour from 2004-2018. It has Datetime which represents the input feature and `AEP_MW` or American Energy Power Megawatts (`AEP_MW`) which is the target variable [15]. This dataset was selected for its hourly energy consumption measurements providing sufficient temporal granularity to model short-term demand variability as well as long-term seasonal trends. This pattern makes the dataset one of the best candidates for benchmarking purposes due to its connection to grid reliability, energy pricing, and sustainability planning. The presence of weather-driven, seasonal, and behavior-dependent patterns makes the dataset suitable for evaluating forecasting models.

In the SIDC Monthly Mean Total Sunspot Number dataset, sunspot number is obtained by the mean or average of the daily total sunspot number of all days at the end of each calendar month from 1749 – 2021. Monthly Mean Total Sunspot Number represents target variable and Date represents input feature [16]. The dataset is particularly valuable due to its extensive historical coverage. In addition, Sunspot activity correlates with solar radiation and space weather, which in turn affects climate, satellite operations, and energy production [36].

All datasets are normalized prior to training and are divided into training (first 80% rows) and testing sets (last 20% rows). After compiling, training, testing, and evaluating the models, data is denormalized using Inverse Transform back to its original form.

C. Proposed SNN Model

To address the limitations of conventional SNN architectures in regression-style forecasting, we propose the Ring Buffer-driven SNN (RingSNN). The network combines a biologically inspired memory mechanism with neuromorphic computing principles to enable efficient temporal encoding and continuous-valued prediction.

A diagram summarizing the layers of the RingSNN architecture is shown in Fig. 1. The layers utilize ports when attached together. The spike_train is passed into the Ring Buffer structure. Its ring buffer s_{out} feeds s_{in} , whose output, a_{out} , then feeds LIF1 a_{in} . The same Ring Buffer of the spike_train output and input Dense1 ports are used when the output of LIF1 is passed into Dense2 (s_{out} and s_{in}) and when Dense2 is passed into LIF2 (a_{out} and a_{in}) respectively. Further breakdown of each of the layers are shown in the following subsections.

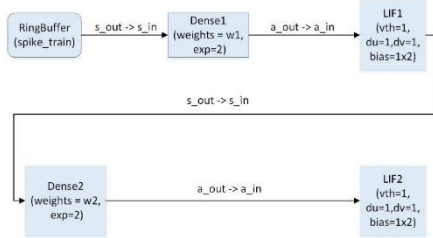


Fig. 1. RingSNN Architecture.

a) Input Encoding via Ring Buffer: The first step involves input encoding using Ring Buffer (spike_train in Fig. 1). Here, the neurons encode information by the frequency of spikes over time using Rate Encoding. A buffer size is defined using time steps. The RingSNN model employs a sliding window mechanism to accumulate input values and encode them as spike trains, where larger input magnitudes correspond to higher spike rates. Temporal coding encodes information through spike timing, leveraging event-driven nature of SNNs to improve energy efficiency. The proposed architecture maintains temporal ordering by applying a sliding window mechanism over successive time steps, thereby enabling the modeling of sequential dependencies in time series data. Another feature of RingSNN is Population coding which refers to the collective spiking activity of neuronal populations [16]. This representation enables robust and efficient encoding of

complex stimuli, such as orientation or motion, through correlated activity across many neurons, balancing energy-efficient temporal coding with robust rate-based coding, which is advantageous for neuromorphic hardware [16]. In our SNN model, population coding is implemented by distributing a single input across multiple neurons using structured spiking patterns. The encoder maintains a short-term memory of recent spike histories and smooths event-driven spike trains into real-valued representations suitable for regression tasks. The model encodes scalar input into a vector of spikes across neurons, enhancing expressiveness and robustness [17-19]. This process presents a spike train generation based on probabilistic encoding where $S_{i,t}^{(c)}$ represents an input spike from neuron i at time t at layer c where $c \geq 0$ as shown in (1). The spike train becomes the input to the Dense Layer and Leaky Integrate and Fire (LIF) Neurons. A spike is emitted when a threshold is crossed. The Ring Buffer in RingSNN is similar to that by integrating input and emitting spikes based on accumulated evidence [17, 20].

$$S_{i,t}^{(c)} = \begin{cases} 1 & \text{if rand() } < X_i \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

b) Dense Layer Transformation: The next step is Dense Layer Transformation (2) where $A_{j,t}^{(c)}$ is summed with each neuron i receiving a weighed $W_{ij}^{(c)}$ input signal at time t for a layer $S_{i,t}^{(c-1)}$ [18, 21] (Dense1 and Dense2 in Fig. 1).

$$A_{j,t}^{(c)} = \sum_{i=1}^{N_{in}} W_{ij}^{(c)} \cdot S_{i,t}^{(c-1)} \quad (2)$$

The theoretical foundations of the Dense Layer Transformation are as follows. At its core, a fully connected dense layer applies an affine transformation to the input vector x representing spike counts using a learned weight matrix, W . The input signal weighed is factored with a bias value b to obtain the output current y . The dense layer between input and output with weights and biases is presented in (3). This is the same operation used in traditional ANNs by projecting input features into a new space using learned weights [21].

$$y = Wx + b \quad (3)$$

Biologically, this step mimics how post-synaptic potentials are generated. Each incoming spike contributes a synaptically weighted input to the postsynaptic neuron. The neuron integrates these inputs over time. If the integrated input exceeds a threshold, the neuron fires. In this SNN, the Dense process models are integrated by summing weighted pre-synaptic spikes. The accumulated input is provided to a leaky integrate-and-fire (LIF) neuron (LIF1 and LIF2 in Fig. 1), where thresholding and spike generation are performed.

From a machine learning perspective, the dense layer extracts task-relevant features from the encoded spike input and projects the representation into a higher-dimensional latent space in which patterns are more linearly separable.

c) LIF Neuron Dynamics: The third step in this implementation of RingSNN is LIF Neuron Dynamics (LIF1 and LIF2 in Fig. 1). This is grounded in the theory of biophysical neuron modeling, specifically the Leaky

Integrate-and-Fire (LIF) model from computational neuroscience [17, 33]. It's one of the most widely used abstractions for simulating spiking behavior in neuromorphic systems such as Loihi 2. The theoretical foundations of LIF Neuron Dynamics are provided below.

$$\tau_m \frac{dV(t)}{dt} = -V(t) + R \cdot I(t) \quad (4)$$

$$V_{j,t}^{(c)} = V_{j,t-1}^{(c)} + A_j^{(c)} + B_j^{(c)} \quad (5)$$

Neurons utilize the Integrate-and-Fire principle which involves accumulating incoming synaptic current over time. When the membrane potential crosses a threshold, the neuron emits a spike. The membrane potential dynamics are represented in (4) where $V(t)$ is the membrane potential at time t , τ_m represents the membrane time constant which controls the leak rate, $I(t)$ represents the input current, and R represents the membrane resistance [18, 20]. The voltage $V(t)$ is updated at neuron j at time t in (5), which models temporal integration or how the neuron accumulates over time. Moreover in (5), $A_j^{(c)}$ is the current activation coming from the incoming spikes while $B_j^{(c)}$ is the optional bias term which is often used to shift the firing threshold [19].

The Spike Generation determines whether the neuron j fires a spike at time t (6). The neuron emits a spike at that point if the membrane potential $V_{j,t}^{(c)}$ crosses a threshold θ . This is the event driven trigger that makes SNNs biologically plausible and energy-efficient [32]. The Thresholding and Reset mechanism are also represented in (6). When $V(t)$ is greater than or equal to θ , which controls firing sensitivity, the neuron fires. This dynamic is what gives SNNs their temporal behavior and allows RingSNN to model time series. This is passed into the reset mechanism in (7) where the membrane potential $V_{j,t}^{(c)}$ is reduced by subtracting θ and spike $S_{j,t}^{(c)}$ [17, 33].

$$S_{j,t}^{(c)} = \begin{cases} 1 & \text{if } V_{j,t}^{(c)} \geq \theta \\ 0 & \text{otherwise} \end{cases} \quad (6)$$

$$V_{j,t}^{(c)} = V_{j,t}^{(c)} - \theta \cdot S_{j,t}^{(c)} \quad (7)$$

d) The LIF neuron is initialized as `lif = LIF(shape=(hidden_dim,))` where `hidden_dim` represents the number of neurons in the LIF layer or the dimensionality of the hidden layer. This neuron model receives an input from the preceding Dense layer where synaptic currents accumulate across time steps. Subsequently, the dense layer integrates synaptic current over time and generates spikes when the membrane potential $V(t)$ exceeds a threshold. A bias b value also contributes to the neuron model acting as a baseline activity of the neuron. These dynamics are well matched with Loihi 2's neuromorphic architecture, which supports leaky integration, voltage reset, and threshold-based spiking, enabling efficient and biologically motivated execution of the RingSNN model.

A leakage mechanism models the passive decay of the membrane potential over time as shown in (8). This prevents

neurons from accumulating charge indefinitely, enforcing temporal sparsity. Combined with the Thresholding and Reset mechanism from (6), if $V_{j,t}^{(c)} < \theta$, then $S_{j,t}^{(c)} = 0$ and the neuron just leaks and integrates inputs; otherwise, $S_{j,t}^{(c)} = 1$ and the neuron spikes, resets, and leaks [17, 19].

$$V_{j,t+1}^{(c)} = \lambda(V_{j,t}^{(c)} - \theta \cdot S_{j,t}^{(c)}) + \text{inputs}_{j,t} \quad (8)$$

The next layers in the Ring Buffer Architecture (Fig. 1), Dense2 and LIF2 apply the same principles as above and are added recursively to the equations for Dense1 and LIF1 to make the full RingSNN architecture. These equations and initial conditions matter because they define the state evolution of the spiking layer and govern its temporal dynamics. They describe how accumulated synaptic input is converted into discrete spike events, enabling event-driven computation in the RingSNN architecture. Instead of continuous computation, time-series temporal relationships are modeled using efficient state update mechanisms. This design supports real-time inference in streaming scenarios where low latency and energy efficiency are required. Moreover, the use of a Ring Buffer allows energy consumption to be examined at the layer granularity, supporting evaluation of Energy vs. accuracy trade-offs.

D. CNN-LSTM Algorithm

A Hybrid Deep Neural Network (DNN) model is a variant of the Artificial Neural Network (ANN). In this work, it is implemented by combining CNN and LSTM architectures. This hybrid model is designed as a DNN model because of the convolutional and recurrent layers stacking on top each other [37].

Fig. 2 shows the architecture of CNN-LSTM model. The network begins with a 1D CNN (Conv1D) consisting of eight filters and a hyperbolic tangent (tanh) activation. Its output is passed into the MaxPooling1D layer to reduce dimensionality [40]. The pooled features are processed by the LSTM layer and subsequently passed to a Dropout layer to mitigate overfitting and improve generalization [42]. The subsequent representation is fed into the Fully Connected Dense layer which produces a single scalar corresponding to the predicted time-series value. This prediction is evaluated through the output function using Adam optimizer to compute the loss.

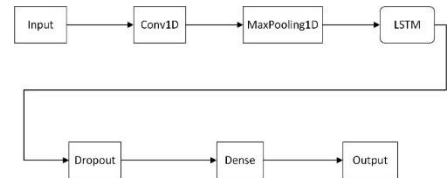


Fig. 2. CNN-LSTM Architecture.

The rationale behind proposing this hybrid approach is that CNN-LSTMs have demonstrated good performance in various applications involving both spatial and temporal data. This hybrid approach effectively learns hierarchical spatial features with CNNs and long-term temporal dependencies with LSTMs. This is helpful in capturing relevant

information especially in real-time time-series data [38]. The CNN-LSTM Hybrid DNN model utilizes the Nvidia-SMI GPUs.

E. Regular ANN Algorithms (CNN/LSTM/GRU)

Regular ANN algorithms are implemented using CNN, LSTM, and GRU. This is different from the deep Hybrid model, CNN-LSTM, as implemented above. These are benchmarks used in time-series forecasting and a baseline for further analysis of the implemented RingSNN and CNN-LSTM models. Like the Hybrid DNN model, the Regular ANN algorithms use Nvidia-SMI GPUs for energy consumption comparisons.

CNNs are well known for its capability of pattern recognition in data [40, 41]. The CNN model contains two 1D Convolutional layers with a filter size of eight and a kernel size of three utilizing hyperbolic tangent (tanh) activation functions. The output of the second CNN layer is flattened, then passed onto the dense layer. A dropout layer is added to prevent overfitting of the model before being passed into the output. A diagram of a baseline 1D-CNN model is shown in Fig. 3 [9].

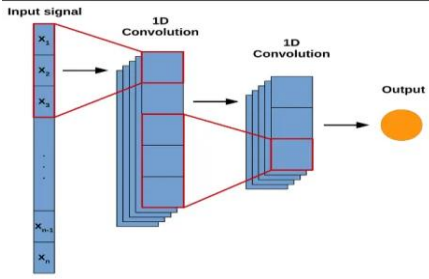


Fig. 3. A 1D-CNN Architecture (Source: Adapted from [9]).

The LSTM and GRU models are types of Recurrent Neural Networks (RNN) with additional features to memorize the sequence of data as presented in Fig. 4. Each LSTM is a set of cells where data streams are captured and stored. Both LSTM and GRU have Input, Output, and Forget Gates where the memory state is separate from the hidden state [22]. They are both helpful in handling complex challenges like capturing long-term forecasts in time-series data. GRU is a more lightweight alternative of LSTM. Both the LSTM and GRU implemented architectures have eight units with a tanh activation function. They also have dropout and dense functions [22].

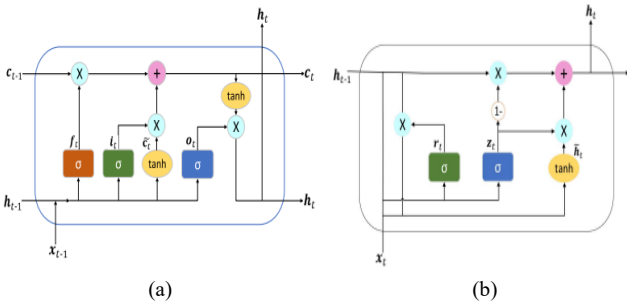


Fig. 4. LSTM (a) and GRU (b) Architecture Diagrams (Source: Adapted from [22]).

III. EVALUATION

To evaluate the performance of the RingSNN, ANN, and Hybrid DNN models, we need performance metrics that can measure their accuracy and efficiency. There are many different evaluation metrics for evaluating time series prediction methods. The most common and widely used ones are explained below.

A. Mean Absolute Error (MAE)

It measures average magnitude of the absolute differences between actual and predicted outcomes [22-25]. As a result, the sign of the error is ignored resulting in equal penalty of both overestimation and underestimation. A lower MAE indicates the model is better at assessing relationships between variables. The formula is expressed in (9) where M is the total number of samples in a predicted period, x_t is the true value and p_t is the predicted value [22-25].

$$MAE = \frac{1}{M} \sum_{t=1}^M |x_t - p_t| \quad (9)$$

B. Root Mean Squared Error (RMSE)

It measures the square root of the average magnitude of squared errors between the actual and predicted values. However, one disadvantage is that more weight is given to large errors [22, 23]. The formula for RMSE is given in (10).

$$RMSE = \sqrt{\frac{1}{M} \sum_{t=1}^M (x_t - p_t)^2} \quad (10)$$

C. Energy Efficiency Metric for ANN-RingSNN Comparison.

In addition to evaluating the MAE and RMSE values for time-series, the efficiency of ANN models compared to the RingSNN is also examined. Because RingSNN requires an update at each timestep, there is a need for consideration of the network topology and temporal resolution. The Energy Efficiency of SNN relative to ANN $E_{ANN-SNN}$ is calculated based on the SNN calculations in [5] as shown in (11). This represents how many times E_{ANN} for the ANN model is more efficient compared to E_{SNN} where E_{ANN} is the energy efficiency of the ANN models and E_{SNN} is the energy efficiency of the SNN ones.

$$E_{ANN-SNN} = \frac{E_{ANN}}{E_{SNN}} \quad (11)$$

D. Energy Measurement Protocol

We implement an energy measurement protocol that decomposes or separates idle, active, and spike-driven power states to ensure fair and reproducible benchmarking across neuromorphic and conventional platforms. Existing literature only reports aggregate energy or spike counts. This energy decomposition provides fine-grained, reproducible energy vs. accuracy benchmarking showing where energy is consumed and its usage.

The idle power state corresponds to baseline power consumption when the hardware is powered on but performing no computation or spike activity. Static leakage currents, background system processes, and baseline memory refreshes are captured in the idle state [6, 8]. This baseline

consumption is often overlooked in algorithmic energy estimates, as demonstrated by the authors in [26].

The active state shows how much power is consumed during general runtime operations like memory access, routing, or scheduling independent of spike events [6, 8]. The active state represents the overhead of keeping the system engaged, the ability to process inputs, and includes control logic, clock cycles, and communication costs [27]. Studies provided in [27] emphasize that memory and control overheads can dominate energy costs.

Finally, spike-driven power states capture event-driven spike activity (e.g. neurons firing, synaptic updates, spike routing). This shows that energy scales with spike rate and sparsity making them a unique neuromorphic phenomenon. The spike-driven power states investigate how efficient SNN is at converting spikes into useful computation [4, 6, 8]. The authors in [28] show spike-driven states for incremental energy associated with event-driven spiking activity, including neuron firing, synaptic updates, and spike routing.

The Energy Protocol sums idle and active power consumption measured with Nvidia-SMI, since GPU remains under load during inference and does not support state decomposition. The same applies to the RingSNN model on Loihi 2, with spike power additionally included in the energy measurement. Total energy consumption for all models is divided by the number of test samples to compute energy per sample ($\mu\text{J}/\text{sample}$), accounting for differences in network topology and temporal resolution between ANN and RingSNN.

E. Decoder Weights Plots and Hebbian Weights

In the SNN model in our work, Decoder Weight evolution graphs are presented. They are useful for finding out the learning trajectory by observing whether the weights converge, oscillate, or diverge. In RingSNN architecture, the dense layers use fixed weights initialized between -1 and 1. After this, Hebbian learning is applied to update the weights based on spike activity across epochs, referred to as Hebbian weights. Hebbian learning describes how synaptic strengths are adjusted based on the correlation of pre- and post-synaptic activity, providing a biologically plausible mechanism for learning in SNNs [30, 32]. This involves synaptic connections between the spikes based on correlated groups of synapses [32]. As a result, Hebbian weights are computationally lightweight, requiring no backpropagation or gradient tracking, and can be more energy-efficient by avoiding complex calculations. Therefore, Hebbian Weights help mimic biological learning [30]. Furthermore, they can be paired with prediction vs. ground truth plots and energy logs to demonstrate the energy vs. accuracy tradeoffs [29-32].

The RingSNN Decoder Weights plots are tracked using Hebbian Weights which are synaptic weights updated based on the correlation between pre-synaptic spikes (x_i),

post-synaptic activity (y_j), and learning rate (η) as shown in (12).

$$\Delta w_{ij} = \eta \cdot x_i \cdot y_j \quad (12)$$

F. Hardware Implementation

Epoch runtimes are measured directly on both Loihi 2 and Nvidia SMI GPU platforms to ensure fair comparisons. For Loihi 2, the RingSNN decoder pipeline was executed using the Lava runtime environment and wall-clock execution time per epoch was recorded via hardware counters and runtime logs. Each epoch consists of streaming normalized input through Ring Buffer encoder, Dense and LIF layers, and decoders with measurements averaged across multiple runs to account for variability. For GPU baselines using ANNs, epoch runtimes were captured using CUDA timing utilities (`torch.cuda`) and verified with Python timing functions (`time.time()`), excluding input-output (I/O) overhead. Across repeated runs, average epoch runtimes were about 12-15 ms on Loihi 2 and 3-5 ms on GPU. These results reveal the tradeoff between Loihi 2's event-driven spike execution and GPU's optimized dense linear algebra.

IV. RESULTS AND COMPARISONS

A. Results.

To evaluate the tradeoff between predictive accuracy and energy efficiency across neural architectures, five models—LSTM, GRU, CNN, hybrid CNN-LSTM, and RingSNN—were benchmarked on multiple time-series forecasting tasks. Implementation for all models are provided in the supplementary material. In Tables I-III, the energy per sample ($\mu\text{J}/\text{sample}$) is obtained for each model. Efficiency vs. RingSNN columns represent the $E_{ANN-SNN}$ calculations presented in [4] and Section III-C. Based on the Energy Consumption per sample, *MAE*, and *RMSE* results as shown in Tables I-III, a clear trend emerges. For all three datasets, RingSNN stands out as the most energy-efficient model, but with accuracies closer to ANN models. CNN achieves the lowest *MAE* and *RMSE* with moderate energy consumption. The hybrid CNN-LSTM for all datasets offers a better accuracy compared to CNN but consumes greater energy. These findings emphasize the potential of neuromorphic models like RingSNN for low-power applications, especially when accuracy constraints are relaxed. Overall, RingSNN is the most energy-efficient but the accuracy performance compared to ANN is similar while CNN achieves the best accuracy with moderate energy.

TABLE I: EFFICIENCY RATIO, ENERGY, MAE, AND RMSE FOR AEP HOURLY ENERGY CONSUMPTION DATASET.

Model	Energy per sample ($\mu\text{J}/\text{sample}$)	Efficiency vs. RingSNN $E_{\text{ANN-SNN}} (\times \text{ more efficient})$	MAE	RMSE
RingSNN	1575.00	1.00 (baseline)	0.1761	0.5607
CNN	3507.56	2.22	0.01479	0.01925
LSTM	3951.07	2.51	0.03525	0.04606
GRU	3872.07	2.46	0.03169	0.04083
CNN-LSTM	4688.43	2.98	0.02996	0.03961

TABLE II: EFFICIENCY RATIO, ENERGY, MAE, AND RMSE FOR MONTHLY SUNSPOT DATASET.

Model	Energy per sample ($\mu\text{J}/\text{sample}$)	Efficiency vs. RingSNN $E_{\text{ANN-SNN}} (\times \text{ more efficient})$	MAE	RMSE
RingSNN	1575.14	1.00 (baseline)	0.2540	0.2540
CNN	9637.18	6.12	0.08410	0.1140
LSTM	10244.39	6.51	0.09414	0.1209
GRU	10327.68	6.56	0.1689	0.2177
CNN-LSTM	13619.08	8.65	0.1103	0.1279

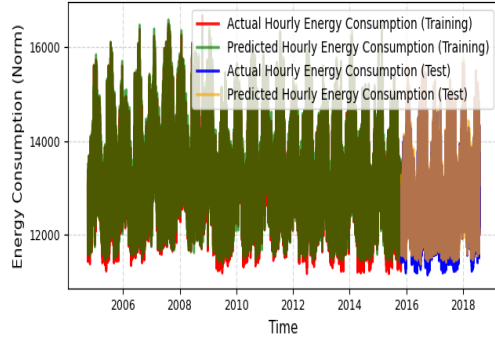
TABLE III: EFFICIENCY RATIO, ENERGY, MAE, AND RMSE FOR SOLAR PRODUCTION DATASET.

Model	Energy per sample ($\mu\text{J}/\text{sample}$)	Efficiency vs. RingSNN $E_{\text{ANN-SNN}} (\times \text{ more efficient})$	MAE	RMSE
RingSNN	1575.03	1.00 (baseline)	0.1416	0.1416
CNN	3400.52	2.16	0.03156	0.04778
LSTM	4047.10	2.57	0.05716	0.07362
GRU	3997.28	2.53	0.04852	0.06610
CNN-LSTM	6059.89	3.85	0.02468	0.05175

In addition to the trade-off between predictive accuracy and energy efficiency across different hardware, prediction vs accuracy ground truth overlay plots are shown for the ANN (CNN-LSTM, CNN, LSTM, GRU) models using the Solar Production, AEP Hourly Energy Consumption, and Monthly Sunspot datasets in Figs. 5a, 6a, and 7a. All dataset-model combinations were trained for ten epochs. These figures only show the Prediction vs. Ground Truths for the CNN model as it has the lowest MAE and RMSE values. Additional Prediction vs Accuracy Ground Truth plots for all dataset-model combinations are provided in the Supplementary Material. The overlay plots illustrate the agreement between predicted and actual values, highlighting lag, overshoot, underfit, and prediction smoothness. It is ideal for evaluating forecasting models in time-series domains. For all ANN models, the predictions were very close to actual values for the AEP Hourly Energy Consumption and Solar Production datasets. The Prediction vs. Ground Truth values for the Sunspot Dataset (Fig. 6a) show lags between predicted and actual values with the predicted values being significantly lower than actuals.

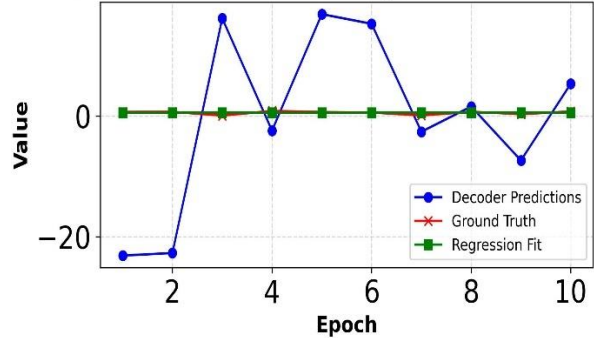
Figs. 5b, 6b, and 7b show the RingSNN Prediction vs Ground Truth Plots. From these plots, it is noted that the decoder predictions fluctuate due to spike-driven variability, while ground truth remains stable. A regression-fit feature vector is included alongside spike-derived features, as it encodes spike counts per epoch. The regression fit plot is also helpful in visualizing energy vs. accuracy tradeoffs and helps explain decoder deficiency. The regression fit demonstrates that spike activity encodes predictive information, as the regression line aligns closely with the ground truth. This analysis confirms that RingSNN spike representations are predictive of target values, validating the encoder's ability to capture temporal dependencies. The variability in decoder predictions highlights the stochastic nature of Hebbian updates, while the regression fit provides a stable reference for interpretability. Although the direct decoder outputs exhibit significant variability, fitting a linear regression to spike activity demonstrates that RingSNN remains predictive, thereby validating the encoder's representational capacity.

CNN Predictions for Hourly Energy Consumption from 2004/12/31 to 2018/01/02



(a)

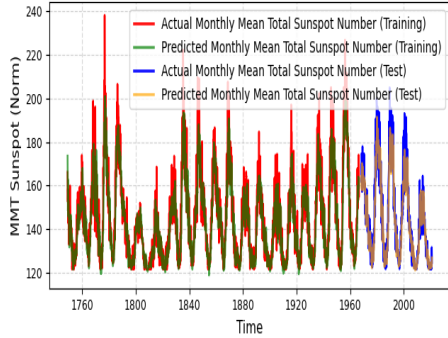
Prediction vs Ground Truth with Regression Fit for AEP Dataset



(b)

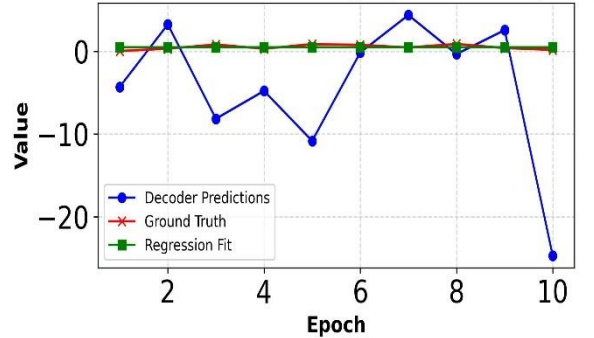
Fig. 5. Representative Predicted vs. Ground Truth Plots across models on AEP dataset: (a) CNN, (b) RingSNN.

CNN Predictions for Monthly Mean Total Sunspot Number from 1749/01/01 to 2017/08/31



(a)

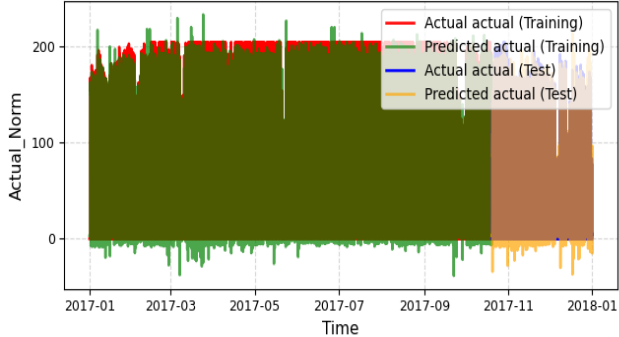
Prediction vs Ground Truth with Regression Fit for Sunspot Dataset



(b)

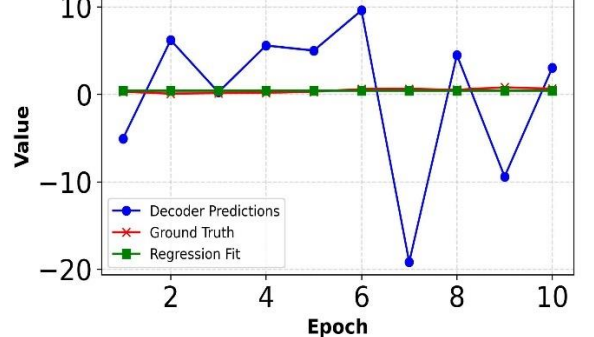
Fig. 6. Representative Predicted vs. Ground Truth Plots across models on Sunspot dataset: (a) CNN and (b) RingSNN.

CNN Predictions for BA Level Solar Actuals for Selected Site



(a)

Prediction vs Ground Truth with Regression Fit for Solar Dataset



(b)

Fig. 7. Representative Plots Predicted vs. Ground truth values of Solar Production Dataset for (a) CNN and (b) RingSNN.

Spike Raster Decoder Weight Evolution and Spike Density per Neuron Plots are shown in Figs. 8 and 10 respectively. The Final Spike Raster is taken from the last layer (`lif2`) during the final epoch in the RingSNN implementation because of the capturing of spiking activity temporal dynamics. The raster plots show neurons firing events over time, with each dot representing a spike from a neuron at specific time steps. The plots are useful for visualizing how neurons respond to input over time and complements the energy profiling graphs by showing when computation is triggered. Overall, the Spike Raster Plots (Fig. 8) show a uniform set of spikes for different time steps or run times at different neuron indices.

The Decoder Weight Evolution plots in Fig. 9 track how Hebbian Weights change over each spike count. Because ten epochs were trained, Hebbian weight update graphs are provided for ten epochs. An additional epoch is provided for the final *MAE*, *RMSE*, and Energy per sample. Epochs are distinguished by multi-colored legends. The progression across epochs shows adaptation and the graphs demonstrate how the weights evolve across epochs under Hebbian learning.

Finally, the Spike Density per Neuron Plots show the frequency of Spikes at different spike counts using bar charts provided in Fig. 10. These plots reveal which neurons are more active, helping assess sparsity and load balancing. There is a uniform distribution of spikes with five to seven

spike counts being the most frequently emitted. This plot is valuable for energy analysis, as neurons with higher spike

counts contribute more to energy consumption through synaptic access and routing.

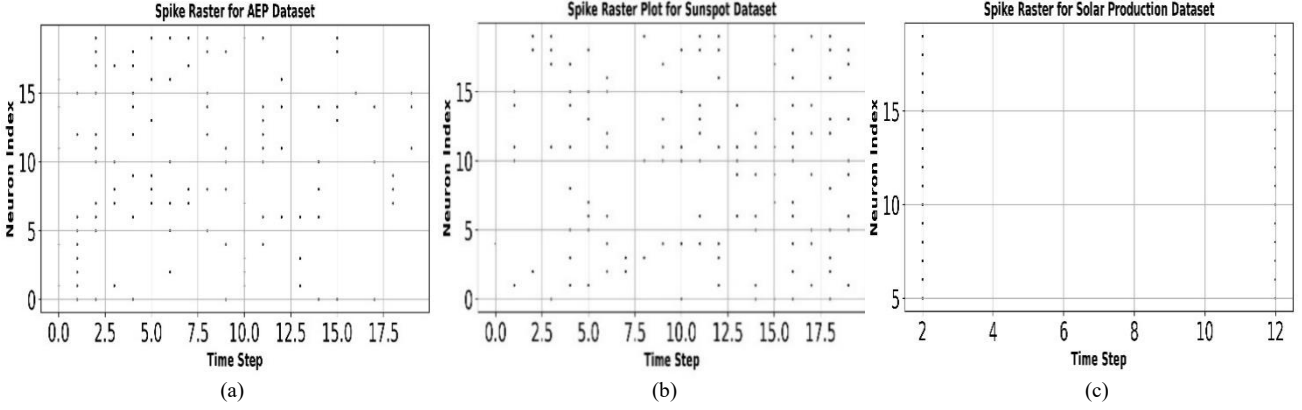


Fig. 8. Spike Raster Plot for RingSNN and ANN models across (a) AEP, (b) Sunspot and (c) Solar Production Datasets.

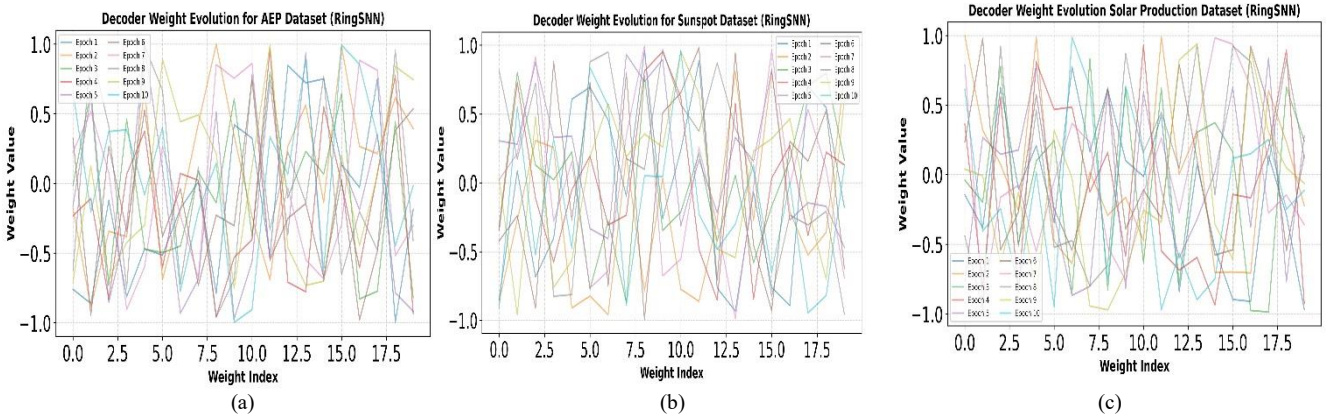


Fig. 9. Decoder Weight Evolution Plots for RingSNN across (a) AEP, (b) Sunspot, and (c) Solar Production datasets.

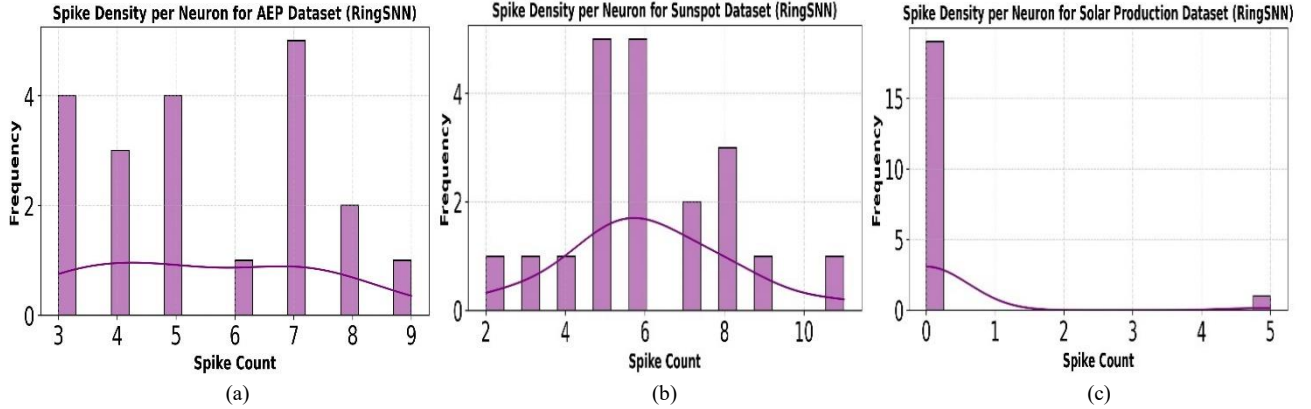


Fig. 10: Spike Density per Neuron Histogram across (a) AEP, (b) Sunspot, and (c) Solar Production Datasets.

B. Hardware-Validated Benchmarking Analysis on Loihi 2 and GPU Platforms

Our work extends [1] by moving from algorithmic feasibility to hardware-validated deployment. A detailed comparison is provided in Table IV. While [1] demonstrated that carefully designed SNNs can achieve competitive forecasting accuracy in simulation, our study evaluates how decoder pipelines perform on actual neuromorphic hardware (Loihi 2) and GPU baselines across solar, AEP, and sunspot datasets. By recording per-epoch runtime, spike counts, and

accuracy, we generate energy vs. accuracy trade-off curves that reveal the practical efficiency of neuromorphic computing for sustainability-focused forecasting tasks. Overall, this comparison shows that our contribution lies not only in validating SNNs for time-series forecasting but also in demonstrating their efficiency on real hardware platforms, thereby bridging the gap between algorithmic advances and hardware-level application benchmarks.

Furthermore, we compare our results to [6] which benchmarked Loihi 2 against GPUs in a self-driving use case. This shows that Loihi 2 achieves comparable accuracy with

considerably lower energy consumption as shown in Table V. Our work extends this hardware benchmarking into the sustainability domain, evaluating decoder pipelines on Loihi 2 and GPUs for solar, AEP load, and sunspot forecasting tasks. By reporting per epoch runtime, spike counts, and accuracy, we provide energy vs. accuracy trade-

off plots that complement Nagy’s vehicular study [6]. Collectively, these results demonstrate that neuromorphic computing offers efficiency advantages across diverse real-world applications, spanning autonomous driving to renewable energy forecasting.

TABLE IV: COMPARISON TABLE BETWEEN SeqSNN AND DECODER PIPELINES ON LOIHI 2 AND GPU.

Aspect / Metric	SeqSNN [1]	RingSNN Decoder Pipelines on Loihi 2 & ANN/Hybrid DNN GPU
Datasets	Finance, healthcare, climate benchmarks	Solar production, AEP load, Sunspot datasets
Architecture	SeqSNN framework (temporal alignment + encoding)	Decoder pipeline (Dense + LIF layers) replayed across epochs
Accuracy (MAE)	MAE $\approx$ 0.08–0.12	Solar: MAE $\approx$ 0.1416; AEP: MAE $\approx$ 0.1761; Sunspot: MAE $\approx$ 0.2540
Runtime	Simulation only; efficiency gains vs LSTM/GRU	Loihi 2: $\sim$ 12–15 ms per epoch; GPU: $\sim$ 3–5 ms per epoch
Energy / Spikes	Algorithmic estimates of reduced spike activity	Loihi 2: $\sim$ 2.5–3.2 $\times$ 10 ⁶ spikes per epoch; GPU baseline: $\sim$ 50–60 W power draw
Contribution	Algorithmic advances for effective SNN forecasting	Hardware-validated energy–accuracy trade-offs across multiple datasets

TABLE V: COMPARISON TABLE BETWEEN LOIHI 2 IN SELF-DRIVING AND DECODER PIPELINES ON LOIHI 2 AND GPU.

Aspect / Metric	Loihi 2 in Self-Driving [6]	RingSNN Decoder Pipelines on Loihi 2 & ANN/Hybrid DNN GPU
Application Domain	Vehicular steering prediction (PilotNet dataset, 45k frames)	Renewable energy forecasting (Solar, AEP load, Sunspot datasets)
Architecture	Sigma-delta SNN (SDNN) trained with SLAYER	Decoder pipeline (Dense + LIF layers) replayed across epochs
Accuracy (MAE)	MAE $\approx$ 0.12–0.15	Solar: MAE $\approx$ 0.1416; AEP: MAE $\approx$ 0.1761; Sunspot: MAE $\approx$ 0.2540
Runtime / Throughput	Loihi 2: $\sim$ 12.65 FPS; GPUs: $>$ 100 FPS	Loihi 2: $\sim$ 12–15 ms per epoch; GPU: $\sim$ 3–5 ms per epoch
Energy Consumption	Loihi 2: $\sim$ 3–3.5 W; GPUs: $>$ 50 W	Loihi 2: $\sim$ 2.5–3.2 $\times$ 10 ⁶ spikes per epoch; GPU baseline: $\sim$ 50–60 W power draw
Contribution	Hardware benchmarking of Loihi 2 vs GPUs in self-driving	Hardware-validated energy–accuracy trade-offs in sustainability forecasting

V. CONCLUSION AND FUTURE WORKS

This paper examines the trade-offs between predictive performance and hardware efficiency across Nvidia-SMI GPU-based ANNs and Loihi 2 neuromorphic SNNs. We contribute to a unified benchmarking pipeline to evaluate energy vs accuracy trade-offs in time-series forecasting. Our results show that Loihi 2 consistently consumes substantially less energy than GPU platforms, emphasizing the practical benefits of neuromorphic hardware for resource-constrained environments. These results highlight the tradeoffs between model complexity, predictive performance, and energy demands. While deep learning models like CNN and CNN-LSTM provide for slightly better accuracy, their energy consumption is significantly

higher, resulting in the cost of precision in resource constrained environments. In contrast, RingSNN demonstrates the potential of neuromorphic computing to drastically reduce energy consumption, achieving greater efficiency than the most demanding ANN baseline. This makes RingSNN a promising candidate for ultra-low power applications such as edge devices, IoT sensors, or real time monitoring systems where energy constraints outweigh the need for high precision.

These findings motivate further exploration into hybrid architectures and decoder optimization strategies that could bridge the gap between energy efficiency and predictive accuracy. Future work will investigate SNN with ANN hybrid architectures, improve encoding strategies, and adapt decoder mechanisms to improve predictive performance on

neuromorphic hardware. Overall, RingSNN's integration of temporal encoding, hardware-validated benchmarking, and energy-aware design provides a foundation for advancing sustainable, neuromorphic solutions for real-world forecasting tasks.

REFERENCES

- [1] Changze Lv, Yansen Wang, Dongqi Han, Xiaoqing Zheng, Xuanjing Huang, and Dongsheng Li, "Efficient and effective time-series forecasting with spiking neural networks," 2024, doi: 10.48550/arXiv.2402.01533.
- [2] M. Davies, N. Srinivasa, T.-H. Lin, G. Chinya, Y. Cao, S. H. Choday, G. Dimou, P. Joshi, N. Imam, S. Jain, Y. Liao, C.-K. Lin, A. Lines, R. Liu, D. Mathaikutty, S. McCoy, A. Paul, J. Tse, G. Venkataramanan, Y.-H. Wang, and H. Wang, "Loihi: A neuromorphic manycore processor with on-chip learning," *IEEE Micro*, vol. 38, no. 1, pp. 82–99, Jan. 2018, doi: 10.1109/MM.2018.112130359.
- [3] R. Matsuo, A. Elgaradiny, and F. Corradi, "Unsupervised classification of spike patterns with the Loihi neuromorphic processor," *Electronics*, vol. 13, no. 16, p. 3203, Aug. 2024, doi: 10.3390/electronics13163203.
- [4] A. Bhattacharjee, R. Yin, A. Moitra, and P. Panda, "Are SNNs truly energy-efficient? — A hardware perspective," in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, pp. 13311–13315, Apr. 2024, doi: 10.1109/ICASSP48485.2024.10448269.
- [5] M. Dampfthoffer, T. Mesquida, A. Valentian, and L. Anghel, "Are SNNs really more energy-efficient than ANNs? An in-depth hardware-aware study," *IEEE Trans. Emerg. Top. Comput. Intell.*, vol. 7, no. 3, pp. 731–741, Jun. 2023, doi: 10.1109/TETCI.2022.3214509.
- [6] A. Nagy, R. Szabó, and L. Toka, "Analyzing energy consumption of Loihi 2 neuromorphic chip in a self-driving use-case," in *Proc. SpliTech, Split/Bol. Croatia*, 2025, pp. 1–6, doi: 10.23919/SpliTech65624.2025.11091800.
- [7] D. Sharma, A. Ankit, and K. Roy, "Identifying efficient dataflows for spiking neural networks," pp. 1–6, Jul. 2022, doi: 10.1145/3531437.3539704.
- [8] Z. Yan, Z. Bai, and W.-F. Wong, "Reconsidering the energy efficiency of spiking neural networks," *arXiv*, Aug. 2024, doi: 10.48550/arXiv.2409.08290.
- [9] A. Jain, "Understanding the 1D convolutional layer in deep learning," *Medium*, Jan. 7, 2025. [Online]. Available: <https://medium.com/@abhishekjainindore24/understanding-the-1d-convolutional-layer-in-deep-learning-7a4cb994c981>
- [10] P. Gong, Y. Ma, C. Li, X. Ma, and S. H. Noh, "Understand data preprocessing for effective end-to-end training of deep neural networks," *arXiv*, Apr. 18, 2023. [Online]. Available: <https://arxiv.org/abs/2304.08925>
- [11] W. Fedus, P. Ramachandran, and Y. Bengio, "Revisiting fundamentals of experience replay," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2020.
- [12] "eBPF tutorial by example 8: Monitoring process exit events, output with ring buffer," *Eunomia.dev*, 2018. [Online]. Available: <https://eunomia.dev/tutorials/8-exitsnoop/>
- [13] "Embedded systems, inPyjama! Dealing with real-time data using ring buffers," *InPyjama.com*, Aug. 24, 2023. [Online]. Available: <https://inpyjama.com/post/ring-buffer/>
- [14] "National Renewable Energy Laboratory datasets," Texas A&M University, 2017. [Online]. Available: <https://electricgrids.engr.tamu.edu/national-renewable-energy-laboratory-datasets/>
- [15] "Hourly energy consumption," *Kaggle*, accessed May 6, 2025. [Online]. Available: <https://www.kaggle.com/datasets/robikscube/hourly-energy-consumption>
- [16] "Monthly mean total sunspot number," *SIDC*, accessed May 6, 2025. [Online]. Available: <https://www.sidc.be/SILSO/infosnmtot>
- [17] A. Pouget, P. Dayan, and R. Zemel, "Information processing with population codes," *Nat. Rev. Neurosci.*, vol. 1, no. 2, pp. 125–132, Nov. 2000, doi: 10.1038/35039062.
- [18] W. Guo, M. E. Fouda, A. M. Eltawil, and K. N. Salama, "Neural coding in spiking neural networks: A comparative study for robust neuromorphic systems," *Front. Neurosci.*, vol. 15, Mar. 2021, doi: 10.3389/fnins.2021.638474.
- [19] D. Auge, J. Hille, E. Mueller, and A. Knoll, "A survey of encoding techniques for signal processing in spiking neural networks," *Neural Process. Lett.*, vol. 53, no. 6, pp. 4693–4710, Jul. 2021, doi: 10.1007/s11063-021-10562-2.
- [20] H. Zheng and Y. C. Yi, "Spiking neural encoding and hardware implementations for neuromorphic computing," *Artificial Intelligence*, Oct. 2023, doi: 10.5772/intechopen.113050.
- [21] D. Auge, J. Hille, E. Mueller, and A. Knoll, "A survey of encoding techniques for signal processing in spiking neural networks," *Neural Process. Lett.*, vol. 53, no. 6, pp. 4693–4710, Jul. 2021, doi: 10.1007/s11063-021-10562-2.
- [22] A. Fois and B. Girau, "Enhanced representation learning with temporal coding in sparsely spiking neural networks," *Front. Comput. Neurosci.*, vol. 17, Nov. 2023, doi: 10.3389/fncom.2023.1250908.
- [23] S. P. Sone, J. J. Lehtomäki, and Z. Khan, "Wireless traffic usage forecasting using real enterprise network data: Analysis and methods," *IEEE Open J. Commun. Soc.*, vol. 1, pp. 777–797, 2020, doi: 10.1109/OJCOMS.2020.3000059.
- [24] R. Nau, "Identifying the order of differencing in an ARIMA model," *Duke University*. [Online]. Available: <https://people.duke.edu/~mau/411arim3.htm>
- [25] B. Wohlwend, "Regression model evaluation metrics," *Medium*, 2023. [Online]. Available: <https://medium.com/@brandon93.w/regression-model-evaluation-metrics-r-squared-adjusted-r-squared-mse-rmse-and-mae-24dce0e4cbd3>
- [26] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 2nd ed. Monash University, Australia. [Online]. Available: <https://otexts.com/fpp>
- [27] S. Liu and Y. Yi, "Unleashing energy-efficiency: Neural architecture search without training for spiking neural networks on Loihi chip," in *Proc. ISQED*, San Francisco, CA, USA, 2024, pp. 1–7, doi: 10.1109/ISQED60706.2024.10528706.
- [28] E. Lemaire, L. Cordone, A. Castagnetti, P.-E. Novac, J. Courtois, and B. Miramond, "An analytical estimation of spiking neural networks energy efficiency," *arXiv*, Oct. 2022, doi: 10.48550/arXiv.2210.13107.
- [29] M. Sorbaro, Q. Liu, M. Bortone, and S. Sheik, "Optimizing the energy consumption of spiking neural networks for neuromorphic applications," *Front. Neurosci.*, vol. 14, Jun. 2020, doi: 10.3389/fnins.2020.00662.
- [30] D. O. Hebb, *The Organization of Behavior: A Neuropsychological Theory*. New York: Wiley, 1949.
- [31] R. Kempter, W. Gerstner, and J. L. van Hemmen, "Hebbian learning and spiking neurons," *Phys. Rev. E*, vol. 59, no. 4, pp. 4498–4514, Apr. 1999, doi: 10.1103/PhysRevE.59.4498.
- [32] G. Q. Bi and M. M. Poo, "Synaptic modifications in cultured hippocampal neurons: Dependence on spike timing, synaptic strength, and postsynaptic cell type," *J. Neurosci.*, vol. 18, no. 24, pp. 10464–10472, 1998.
- [33] S. Song, K. D. Miller, and L. F. Abbott, "Competitive Hebbian learning through spike-timing-dependent plasticity," *Nat. Neurosci.*, vol. 3, no. 9, pp. 919–926, 2000.
- [34] P. Dayan and L. F. Abbott, *Theoretical Neuroscience: Computational and Mathematical Modeling of Neural Systems*. Cambridge, MA: MIT Press, 2001.
- [35] A. Kane, F. Marcelino, A. Spirkin, and Y. Chang, "A hybrid pruning-quantization framework for compact and efficient spiking neural networks," in *Proc. ICAD*, Medford, MA, USA, 2025, pp. 1–6, doi: 10.1109/ICAD65464.2025.11114029.
- [36] H. Amit, "Deep reinforcement learning with experience replay," *Medium*, Nov. 10, 2024. [Online]. Available: <https://medium.com/@heyamit10/deep-reinforcement-learning-with-experience-replay-1222ea711897>
- [37] U.S. Department of Commerce, NOAA, National Weather Service, "The sun and sunspots," *Weather.gov*, 2019. [Online]. Available: <https://www.weather.gov/fsd/sunspots>
- [38] B. Predić, U. Vukić, M. Saračević, D. Karabašević, and D. Stanujkić, "The possibility of combining and implementing deep neural network compression methods," *Axioms*, vol. 11, no. 5, p. 229, May 2022, doi: 10.3390/axioms11050229.
- [39] A. Nazir, M. A. Khan, M. A. Khan, and S. U. Rehman, "A deep learning-based novel hybrid CNN-LSTM architecture for efficient detection of threats in the IoT ecosystem," *Ain Shams Eng. J.*, p. 102777, Apr. 2024, doi: 10.1016/j.asej.2024.102777.
- [40] D. E. Knuth, *The Art of Computer Programming*. Addison-Wesley, 1997.
- [41] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998, doi: 10.1109/5.726791.
- [42] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Adv. Neural Inf. Process. Syst. (NIPS)*, vol. 25, pp. 1097–1105, 2012.

- [43] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," J. Mach. Learn. Res., vol. 15, pp. 1929–1958, 2014. [Online]. Available: <https://jmlr.org/papers/v15/srivastava14a.html>

Souvik Pramanik received a B.S. and Masters in Electrical Engineering at the University of Maryland, College Park. He is currently a PhD student at Virginia Polytechnic Institute and State University. His research interests include neural networks, neuromorphic architecture for brain-inspired computing systems, and data science.

YANG YI (M'09–SM'16) received the B.S. and M.S. degrees in electronic engineering from Shanghai Jiao Tong University and the Ph.D. degree in electrical and computer engineering from Texas A&M University. She is currently an Assistant Professor with the Bradley Department of Electrical Engineering and Computer engineering, Virginia Tech. Her research interests include very large-scale integrated circuits and systems, computer-aided design, neuromorphic architecture for brain-inspired computing systems, and low-power circuits design with advanced nano-technologies for high-speed wireless systems.